

Lab 2 Report

By: Sunny Kang, Tracy Yang

April 19, 2023

ECE 2049 D Term 2023

Group 3, Board 24

Introduction

By coding Guitar Hero onto the MSP430, we successfully practiced coding for I/O functions. We did so through buttons, a buzzer, the LCD, the LEDs, and a timer. From this lab, we were able to learn how to code functions that depend on timers as well as grasp an in-depth understanding on how timers work.

Discussion and Results

The first task into creating Guitar Hero was figuring out how to play the song. We determined that arrays were the best data structure to store the pitch and duration. We also stored the corresponding LED in an array as well.

Once that was figured out, we needed to move onto the switch cases. We have the following cases: the welcome screen, the countdown, and the game itself. When the board is turned on, the LCD displays “WELCOME TO GUITAR HERO press (*) to start.” After the asterisk is pressed, we implemented a countdown that uses the timer. The board switches the number on the screen and lights up its LEDs by comparing the global timer to a local timer that we created a variable for. The local time is documented in the previous state so that the global timer has something to compare, and if the difference between the time is a specific number, it'll clear the LCD screen and display the countdown. Though this method of timing the code was useful, we eventually had our final code use a mix of interrupts and polling as it was unnecessary to have simple functions use the timer. For example, in our resetting code, we used a delay that flashed “Resetting!” then transitioned back to the welcome screen.

After the countdown was complete, we moved onto the game. We firstly determined the conversion of frequency to Timer B CCR0 tics using the formula below.

$$\frac{1}{\text{note frequency}} = \text{period} = (\text{max_count} + 1) \frac{1}{32768}$$

Plugging in the frequency values of each pitch, we then get the timer B values, which we decided to use an octave of the c major scale. Then, we moved onto playing the song using the timer rather than delays. In the main game loop, we used a for loop to keep track of the place in the array and the while loop to compare the global time to the local time plus the duration (which depended on which element we are in the array). Additionally, by initiating the button as 0, it allowed for a true condition to read what buttons are being pressed. We did so because button was redefining itself within the while loop, and the if condition resolved that issue.

```
case 2:
    Graphics_clearDisplay(&g_sContext);
    runTimerA2();
    for (i = 0; i < 32; i++){
        Graphics_clearDisplay(&g_sContext);
        button = 0;
        local_time1 = timer_cnt;
        while (timer_cnt < local_time1 + duration[i]*75){
            if (button == 0){
                button = readButtons();//gets the button user pressed
                Graphics_flushBuffer(&g_sContext);
            }
            BuzzerOn(note[i]);
            setLeds(Led[i]);
        }
    }
}
```

Figure 1:

Figure 1 shows the code that plays the song and takes in the button inputs.

When we experimented with software delays and the timer together, we noticed that the delays would mess up the timings. Because delays are a blocking function, the board cannot check the user's input when the note is playing. The timer, on the other hand, allows us to do things simultaneously. We also determined that Timer A2 should be 163 ACLK ticks, making the resolution 0.005 seconds. This is because the error increases as the resolution increases, which in turn increases the leap count. By having the resolution several times smaller than the duration of a note, it minimizes the error.

Lastly, the scoring for points in the game is as follows: for every button pressed correctly, 1 point is earned. Incorrect inputs get a strike, and when the player gets 3 strikes, they lose. Additionally, if the player misses any notes, they lose.

Summary and Conclusion

From lab 2, we learned how to use the timer and the buzzer as well as continued to practice coding switch cases. We were also able to learn how to create functions that relied on timing. Peripheral functions like BuzzerOn, readButtons, SetUserLED familiarized us more with the board, and it successfully enhanced the application of what we learned in lecture.

Appendix

```
void configButton(void)
{
    //S1 P7.0
    P7SEL &= ~(BIT0);
    P7DIR &= ~(BIT0);
    P7REN |= (BIT0);
    P7OUT |= (BIT0);

    //S2 P3.6
    P3SEL &= ~(BIT6);
    P3DIR &= ~(BIT6);
    P3REN |= (BIT6);
    P3OUT |= (BIT6);

    //S3 P2.2
    P2SEL &= ~(BIT2);
    P2DIR &= ~(BIT2);
    P2REN |= (BIT2);
    P2OUT |= (BIT2);

    //S4 P7.4
    P7SEL &= ~(BIT4);
    P7DIR &= ~(BIT4);
    P7REN |= (BIT4);
    P7OUT |= (BIT4);
    P7IN &= 0x00;
}
```

Figure:

Figure shows the prelab for configuring the buttons.

```
unsigned char readButtons(void)
{
    unsigned char button = 0;

    if ((P7IN & BIT0)==0) //S1
        button |= BIT3;
    else
        button &= ~BIT3;

    if ((P3IN & BIT6)==0) //S2
        button |= BIT2;
    else
        button &= ~BIT2;

    if ((P2IN & BIT2)==0) //S3
        button |= BIT1;
    else
        button &= ~BIT1;

    if ((P7IN & BIT4)==0) //S4
        button |= BIT0;
    else
        button &= ~BIT0;

    return button;
}
```

Figure:

Figure shows the function that returns the state of the buttons.

```

void BuzzerOn(unsigned char note)
{
    // Initialize PWM output on P3.5, which corresponds to TB0.5
    P3SEL |= BIT5; // Select peripheral output mode for P3.5
    P3DIR |= BIT5;

    TB0CTL = (TBSSEL__ACLK|ID__1|MC__UP); // Configure Timer B0 to use ACLK, divide by 1, up mode
    TB0CTL &= ~TBIE; // Explicitly Disable timer interrupts for safety

    // Now configure the timer period, which controls the PWM period
    // Doing this with a hard coded values is NOT the best method
    // We do it here only as an example. You will fix this in Lab 2.

    if (note == 'c')
        TB0CCR0 = 63; // Set the PWM period in ACLK ticks
    if (note == 'd')
        TB0CCR0 = 56;
    if (note == 'e')
        TB0CCR0 = 50;
    if (note == 'f')
        TB0CCR0 = 47;
    if (note == 'g')
        TB0CCR0 = 42;
    if (note == 'a')
        TB0CCR0 = 37;
    if (note == 'b')
        TB0CCR0 = 35;
    if (note == 'C')
        TB0CCR0 = 31;

    TB0CCTL0 &= ~CCIE; // Disable timer interrupts

    // Configure CC register 5, which is connected to our PWM pin TB0.5
    TB0CCTL5 = OUTMOD_7; // Set/reset mode for PWM
    TB0CCTL5 &= ~CCIE; // Disable capture/compare interrupts
    TB0CCR5 = TB0CCR0/2; // Configure a 50% duty cycle
}

```

Figure:

Figure shows the configuration of notes onto the buzzer.

```

unsigned char note[32] = {'a', 'c', 'd', 'd', 'b', 'a', 'd', 'd', 'b', 'b', 'a', 'g', 'f', 'c', 'a', 'c', 'd', 'd', 'b', 'a', 'd', 'd', 'b', 'a', 'g', 'f', 'g', 'e', 'f', 'g', 'f'};
unsigned char duration[32] = {4, 2, 1, 5, 2, 2, 4, 1, 3, 1, 2, 2, 2, 9, 4, 2, 1, 5, 2, 2, 4, 1, 3, 1, 2, 2, 9, 2, 2, 2, 8};
unsigned char led[32] = {4, 8, 4, 4, 2, 4, 4, 2, 2, 4, 8, 1, 8, 4, 8, 4, 4, 2, 4, 4, 4, 2, 2, 4, 8, 1, 8, 2, 1, 8, 1};

```

Figure:

Figure shows the arrays of the pitch, duration, and LEDs.